Climate Analytics Platform Technical Series

A deep-dive technical series on the technical stack behind this project's analysis, API, dashboards and conversational agent. Each area explains the **concepts**, the **technical approach and implementation**, **testing**, and **deployment** best practices. The examples are project-agnostic, but chosen to explain the techniques this project actually uses. For what the platform's results mean for climate, see the companion Functional Series.

Start here: Architecture Overview

Architecture Overview is the map. It explains the general shape of a data-science project with visualization dashboards, the architecture principles behind it (contracts, fail-closed serving, provenance, bounded autonomy, design tokens, edge access control), the options at each layer, and how the layers are deployed and operated together. Read it first; every area below assumes it.

How the areas fit together

#	Area	What it covers	Typical order
1	Data Science and ML	EDA, features, regression, forecasting, multi-source pipelines	Produces the data
2	Python API Backend	FastAPI, API design, testing, deployment	Serves the data
3	Streamlit Dashboards	Single-process dashboards	Option A for presenting it
4	React Front End	SPA, routing, design system, dashboard patterns	Option B for presenting it
5	MCP Servers	Exposing the API as tools for an LLM	Optional layer
6	Conversational Agents	Graph-based agent, guardrails, generative UI	Optional layer on top of 5

Each area is independent and can be worked through on its own; the areas link to one another where one builds on another. Each document ends with its own **Core Libraries** summary and a **See also** list.

Data Science and ML

Take a raw dataset all the way to a compared, validated set of models and a multi-period forecast — and, for products that keep several external sources current, a validated, provenance-tracked pipeline.

Reading order

1. **EDA & Data Engineering** — profiling a raw dataset, filtering it responsibly, exploratory data analysis, and the data-quality discipline needed before any modeling begins.
2. **Feature Engineering** — turning cleaned data into model-ready inputs: time-based, lag, rolling, growth, and ratio features; scaling; categorical encoding.

3. **Regression Models** — Naive baseline, Linear Regression, and Random Forest: the concepts, the scikit-learn API, key parameters, evaluation metrics, and the bias-variance trade-off that ties them together.
4. **Time-Series Forecasting** — Exponential Smoothing (ETS/Holt's Damped Trend), multi-step recursive forecasting, and what-if scenario modeling, plus where this fits relative to ARIMA and other forecasting approaches.
5. **Multi-Source Pipelines** — what changes when several external sources are combined and refreshed on a schedule: one ingestion module per source, validation before publishing, provenance and licence as data, a harmonised layer, honest statistical relationships (HAC errors, block bootstrap, holdouts), scenario translation, and unattended operation.

Documents 1–4 are the core curriculum, built on one dataset; document 5 is the extension for a product that has to keep several sources current. Each document ends with its own **Core Libraries**, **Testing & Validation**, and (where relevant) **Deployment** sections, so it can be read and referenced independently once you've been through the series once.

Prerequisites

Comfort with Python and basic pandas (reading a CSV, indexing a DataFrame) is assumed. No prior machine learning or statistics background is assumed — each concept is introduced before the code that implements it.

See also

- Architecture Overview for how this pipeline's output typically gets served and displayed
- Python API Backend for how to expose these results over HTTP

Python API Backend

Build a Python web API that serves computed analysis results as JSON — the decoupled presentation-layer option described in the architecture overview.

Reading order

1. **HTTP, REST & FastAPI Fundamentals** — HTTP methods and status codes, REST conventions, and the FastAPI framework's routing, validation, and dependency-injection model.
2. **API Design & Best Practices** — data loading and caching, error-handling patterns (failing closed on pipeline output), resource/URL design, versioning, pagination, response envelopes, reporting the scope actually served, security fundamentals.
3. **Testing** — `pytest`, FastAPI's `TestClient`, fixture-based test data, mocking, fixtures built by running the real upstream pipeline, and how to verify a test actually catches the regression it claims to.
4. **Deployment** — running the API in production: ASGI servers, containers, cloud platforms, reverse proxies and edge policies, refresh-driven restarts, CI/CD, and health checks.

Prerequisites

Comfort with Python functions, classes, and type hints is assumed. No prior web-framework experience is assumed.

See also

- Architecture Overview for how this layer fits between the data-processing layer and a frontend
- Data Science / ML for what typically produces the data this API serves
- React Front End for a typical human-facing consumer of an API built this way
- MCP Server for wrapping this same API as tools an LLM agent can call, instead of (or alongside) a frontend

Streamlit Dashboards

Build a quick, single-process interactive dashboard — the "Option A" presentation-layer approach in the architecture overview.

Reading order

1. **Core Concepts** — the rerun-on-every-interaction mental model, caching, widgets, layout, session state, and forms.
2. **Charting & Visualization** — Plotly Express and Graph Objects in depth, plus other charting options worth knowing about.
3. **Testing** — Streamlit's `AppTest` framework, and a candid discussion of when manual verification is a reasonable alternative.
4. **Deployment** — Streamlit Community Cloud, containerized deployment, and secrets management.

Prerequisites

Comfort with Python and basic pandas is assumed. No prior web-framework experience is assumed — Streamlit is deliberately designed not to require one.

See also

- Architecture Overview for how this approach compares to a decoupled API + frontend
- Data Science / ML for what this kind of dashboard typically reads and displays
- React Front End for the alternative, decoupled approach

React Front End

Build a decoupled, component-based frontend that consumes a backend API — the "Option B" presentation-layer approach in the architecture overview.

Reading order

1. **Core Concepts** — components, props, state, hooks, the custom-hook pattern for data fetching, and how a design system (tokens, themes, components, tests) is structured and consumed.
2. **Routing & API Integration** — client-side routing, a typed API client, mirroring backend response shapes, state-management options beyond local component state, and build-tooling concerns.
3. **Testing** — Vitest/Jest, React Testing Library, mocking, testing custom hooks, testing a data-heavy dashboard's patterns, and where end-to-end testing fits.

4. **Deployment** — static hosting, containerized deployment, CI/CD for a frontend build, service-worker caching and edge-gated routes, and the case where the build itself is the release.
5. **Patterns for Data-Heavy Dashboards** — view-model builders, independent sections, URL-backed state, a page-wide shared control, phone-sized variants, accessibility, theming, and embedding a conversational agent's answers.

Prerequisites

Comfort with modern JavaScript (functions, array methods, `async`/`await`, ES modules) is assumed. TypeScript examples are used throughout, but prior TypeScript experience is not required — the type annotations are explained as they're introduced.

See also

- Architecture Overview for how this compares to a single-process dashboard approach
- Python API Backend for the backend this frontend typically consumes
- Data Science / ML for what the underlying data being visualized typically represents
- Conversational Agents for surfacing an open-ended, natural-language view alongside the fixed charts/controls this curriculum builds

MCP Servers

Build an MCP server — a standard-protocol layer that exposes an existing data source or API as tools an LLM can call.

Reading order

1. **Core Concepts** — what MCP is and the problem it solves, the host/client/server roles, the three primitives (tools, resources, prompts), transports, and a minimal working server.
2. **Tool Design** — designing tool schemas for an LLM audience: direct wraps vs. composed tools, argument-resolution guards, shaping responses for a model rather than a human, error handling, statelessness, interpretive framing, envelope pass-through with computed summaries, and rejecting what a tool cannot honor.
3. **Testing** — unit-testing tool functions directly, integration-testing through a real client, fixture data patterns, a class of entry-point bug only a real subprocess launch can catch, and registry-wide tests for rejection rules.
4. **Deployment** — transports in production, per-boundary auth decisions (a worked four-boundary example with edge service tokens), connecting local, desktop-app and programmatic clients, and independent versioning.

Prerequisites

Comfort with Python functions, classes, and type hints is assumed. Having read the Python API Backend curriculum first is helpful but not required — an MCP server commonly wraps a REST API like the one built there, and several documents in this folder point back to the equivalent backend concept for comparison.

See also

- Architecture Overview for how this layer fits between a data-serving API and an LLM agent
- Python API Backend for a typical thing this kind of server wraps
- Conversational Agents for the typical consumer of an MCP server's tools

Conversational Agents

Build a graph-based conversational agent — an LLM given tools and the autonomy to decide when to use them, typically calling an MCP server and surfaced through a frontend.

Reading order

1. **Core Concepts** — what "agent" means here, why a graph rather than a plain loop, nodes/edges/conditional routing, state schema and reducers, the tool-calling loop, and checkpointer-backed memory.
2. **Tool Calling, Guardrails and Generative UI** — using an MCP server as a tool source, classifying requests before acting on them, bounding the tool-calling loop, two different kinds of caching, streaming progress, mapping results to real UI components instead of freeform generated markup, and structural guardrails (required disclosures, capped model views, computed numbers, fixed follow-up lookups).
3. **Testing** — an injectable LLM seam for hermetic tests, stub-based graph-routing tests, testing reducers against a real graph invocation, real-subprocess integration testing of the tool connection, exactly one gated live-LLM smoke test, table-driven tests for the deterministic parts, and answer linting with a golden-prompt evaluation.
4. **Deployment** — trust boundaries between an agent and its tool source, session-id validation as a public input boundary, secrets, streaming responses, rate limiting, a safe runtime model-switch (admin) capability, and observability.

Prerequisites

Comfort with Python is assumed. Having read the MCP Server curriculum first is helpful but not required — this curriculum treats an MCP server as the typical source of an agent's tools, and several documents here point back to the equivalent MCP-side concept.

See also

- Architecture Overview for how this layer fits alongside a more conventional dashboard presentation layer
- MCP Server for the typical tool source this kind of agent calls
- React Front End for a typical UI shell a conversational agent is surfaced through